O'REILLY®

2nd Edition

Designing Data-Intensive Applications

The Big Ideas Behind Reliable, Scalable, and Maintainable Systems

Martin Kleppmann
& Chris Riccomini

Praise for *Designing Data-Intensive Applications*

Designing Data-Intensive Applications has become the Bible of distributed systems—almost every serious practitioner I know owns a copy.

—Will Wilson, CEO at Antithesis

Mastering trade-offs is essential to solving real-world problems with distributed data systems. Designing Data-Intensive Applications explores them like none other, providing an unbiased view of how different systems have made these choices over time.

—Veena Basavaraj, head of application platform engineering at Benchling

An essential guide for distributed systems engineers—thorough and insightful for both beginners and experts.

—Zhengliang "Zane" Zhu, distributed systems engineer, Netflix

This book is a gateway to distributed systems. It's the best-in-class book for getting up to speed on concepts every systems engineer should know.

—Alex Petrov, author of *Database Internals*, Apache Cassandra committer, and PMC member

For the last decade, this has been the best book for understanding distributed systems, and the second edition makes it even better. It bridges the huge gap between distributed systems theory and practical engineering. I wish it had existed when I started working on distributed systems so I could have read it then and saved myself a lot of mistakes.

—Jay Kreps, creator of Apache Kafka and cofounder of Confluent

This book should be required reading for software engineers. Designing Data-Intensive Applications is a rare resource that connects theory and practice to help developers make smart decisions as they design and implement data infrastructure and systems.

—Kevin Scott, Chief Technology Officer at Microsoft

Designing Data-Intensive Applications

The Big Ideas Behind Reliable, Scalable, and Maintainable Systems

SECOND EDITION

Martin Kleppmann and Chris Riccomini

Designing Data-Intensive Applications

by Martin Kleppmann and Chris Riccomini

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (https://oreilly.com). For more information, contact our corporate/institutional sales department: 800-998-9938 or *corporate@oreilly.com*.

Acquisitions Editor: Aaron Black

Development Editor: Melissa Potter

Production Editor: Katherine Tozer

Copyeditor: Rachel Wheeler

Proofreader: Sharon Wilkey

Indexer: Potomac Indexing, LLC

Cover Designer: Susan Brown

Cover Illustrator: Monica Kamsvaag

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

March 2017: First Edition

February 2026: Second Edition

Revision History for the Second Edition

- 2026-02-18: First Release

See https://oreilly.com/catalog/errata.csp?isbn=9781098119065 for release details.

978-1-098-11906-5

[LSI]

Dedication

To everyone using technology and data to address the world's biggest problems.

> *Computing is pop culture. […] Pop culture holds a disdain for history. Pop culture is all about identity and feeling like you're participating. It has nothing to do with cooperation, the past or the future—it's living in the present. I think the same is true of most people who write code for money. They have no idea where [their culture came from].*
>
> —Alan Kay, in interview with *Dr. Dobb's Journal* (2012)

Preface

There are hundreds of databases to choose from. Which one should you use for your application? The short answer is, "It depends." The long answer is…this book.

Different technologies for storing and processing data make different trade-offs, and no one approach is best for all situations. The system that is a perfect fit for one application is badly suited to another. This book is a guide to the entire landscape of data systems, not just looking at one product, but comparing the strengths and weaknesses of many systems.

Although the landscape of technologies for processing and storing data is diverse and fast-changing, the underlying principles endure. If you understand those principles, you're in a position to see where each tool fits in, how to make good use of it, and how to avoid its pitfalls. This book focuses on those principles.

While this book is not a tutorial for using one particular tool, it isn't a textbook full of dry theory either. Instead, we will look at many examples of successful data systems: technologies that form the foundation of numerous popular applications and that have to meet scalability, performance, and reliability requirements in production every day. We will dig into the internals of those systems, tease apart their key algorithms, and discuss the trade-offs they have made. On this journey, we will try to find useful ways of *thinking about* data systems—not just *how* they work, but also *why* they work that way.

After reading this book, you will be in a great position to determine which kinds of technologies are appropriate for which purposes and to understand how tools can be combined to form the foundation of a solid application architecture. You will develop a strong intuition for what your systems are doing

under the hood so that you can reason about their behavior, make good design decisions, and track down any problems that may arise.

The guiding philosophy of this book is to bring together a broad range of perspectives: both theoretical and practical, both recent and old. The computing industry tends to be attracted to things that are new and shiny and to look down on ideas that are considered old or academic. That is a mistake; many powerful, foundational ideas in computing arose from research, some recent, some decades ago. On the other hand, academia sometimes lacks a clear idea of what issues are important in practice. This book combines the best of both: academic precision and attention to detail with an industrial focus on practicality.

Who Should Read This Book?

If any of the following are true for you, you'll find this book valuable:

- You're a software engineer, software architect, or technical manager who needs to make decisions about the architecture of the systems you work on—for example, you need to choose tools for solving a given problem and figure out how best to apply them. This applies especially to backend systems.
- You're a data engineer who wants to understand the wider context of the systems you deal with, or a cloud engineer who wants insights into the underpinnings of the systems you're using. You will find that even though modern distributed systems hide a lot of complexity from you, understanding their underlying principles is extremely useful for performance optimization and debugging.
- You want to learn how to make data systems scalable (e.g., to support apps with millions of users), highly available (minimizing downtime), operationally robust, and easier to maintain in the long run (even as they grow and as requirements and technologies change).
- You are preparing for a "system design" job interview in which you will be asked to sketch an architecture for an application, and you need to learn the principles for good data architectures.
- You are curious to find out what goes on behind the scenes at major websites and online services, and inside various databases and data processing systems—especially if you like to dig deeper than buzzwords to gain a technically accurate and precise understanding of various technologies and their trade-offs.

This book assumes that you already have some experience building web-based applications and that you are familiar with relational databases and SQL. A high-level understanding of common network protocols like TCP and HTTP is helpful. Your choice of programming language or framework makes no difference for this book.

What's New in the Second Edition?

This second edition has the same goals and scope as the first edition of *Designing Data-Intensive Applications*, which was published in 2017. However, we have thoroughly revised the entire book to reflect technological changes that have happened in the last decade and to improve the clarity of the explanations.

The biggest technical changes that have affected this book since the first edition are the explosion of interest in AI and the rise of cloud native data systems architectures. While this book is not about AI per se, we have added coverage of data systems that support AI and machine learning, including vector indexes (used for semantic search), DataFrames (used for training datasets), and batch processing systems for preparing large amounts of training data. Cloud native ideas, such as building data systems on top of object stores instead of local disks, have been woven in throughout the book.

We have also added discussions of sync engines and local-first software, workflow engines and durable execution, formal methods and randomized testing, GraphQL, and various other technologies that are worth knowing about. We have included a bit of legal context as well, by exploring the impact of the EU General Data Protection Regulation (GDPR) and related law. We've also taken a few things away—for example, as MapReduce is now largely obsolete, we have rewritten the batch processing chapter accordingly, and we sadly decided to drop the Tolkien-style maps.

A few discussions have been restructured, and the chapter numbering has changed. Some chapters required only a light edit, while others (such as Chapter 10, on consistency and consensus) were almost completely rewritten to make them clearer. Overall, the second edition is about 60 pages longer than the first.

References and Further Reading

Most of what we discuss in this book has already been said elsewhere in some form or another—in conference presentations, research papers, blog posts, code, bug trackers, mailing lists, or engineering folklore. This book summarizes the most important ideas from many sources, and it includes pointers to the original literature throughout the text. The references at the end of each chapter are great resources if you want to explore an area in more depth, and most of them are freely available online.

In the ebook editions we have included links to the full text of online resources. As links tend to break frequently because of the nature of the web, we have also included archival links where possible. If you come across a broken link, or if you are reading a print copy of this book, you can use a search engine to look up references. For academic papers, search for the title in Google Scholar to find open-access (free) PDF files. Books might cost money, but you shouldn't have to pay for research papers.

Alternatively, you can find all the references at *https://github.com/ept/ddia2-references*, where we maintain up-to-date links.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, datatypes, environment variables, statements, and keywords.

`Constant width bold`

Shows commands or other text that should be typed literally by the user.

`Constant width italic`

Shows text that should be replaced with user-supplied values or by values determined by context.

> **NOTE**
>
> This element signifies a general note.

> **WARNING**
>
> This element indicates a warning or caution.

O’Reilly Online Learning

> **NOTE**
>
> For more than 40 years, O’Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O’Reilly’s online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O’Reilly and 200+ other publishers. For more information, visit *https://oreilly.com*.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

https://oreilly.com/about/contact.html

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at *https://oreil.ly/DesigningDataIntensiveApps2*.

For news and information about our books and courses, visit *https://oreilly.com*.

Find us on LinkedIn: *https://linkedin.com/company/oreilly*.

Watch us on YouTube: *https://youtube.com/oreillymedia*.

Acknowledgments

This book collects and systematizes knowledge and ideas from a huge number of people. It contains about a thousand references to articles, blog posts, talks, documentation, and more, and we are very grateful to the authors of this material for sharing their knowledge.

For the second edition, Chris Riccomini joined Martin Kleppmann as coauthor. We would like to thank everyone who provided feedback and input that improved the book: in particular, David Booth, Mark Callaghan, Chuck Carman, Aniruddh Chaturvedi, William Dealtry, Arbel Deutsch Peled, Phil Eaton, Joy Gao, Johannes Hauser, Matthew Hertz, Erin R. Hoffman, Matt Housley, Karan Johar, Ling Mao, Pedram Navid, Mohit Palriwal, Alex Petrov, Alex Power, Joe Reis, Jack Vanlightly, Will Wilson, Jacky Zhao, and Zhengliang Zhu. Of course, we take all responsibility for any remaining errors or unpalatable opinions in this book.

While he was writing the first edition, Martin Kleppmann benefited from a large number of people who took the time to discuss ideas or patiently explain things to him. He would like to thank Joe Adler, Ross Anderson, Peter Bailis, Márton Balassi, Alastair Beresford, Mark Callaghan, Mat Clayton, Patrick Collison, Sean Cribbs, Shirshanka Das, Niklas Ekström, Stephan Ewen, Alan Fekete, Gyula Fóra,

Camille Fournier, Andres Freund, John Garbutt, Seth Gilbert, Tom Haggett, Pat Helland, Joe Hellerstein, Jakob Homan, Heidi Howard, John Hugg, Julian Hyde, Conrad Irwin, Evan Jones, Flavio Junqueira, Jessica Kerr, Kyle Kingsbury, Jay Kreps, Carl Lerche, Nicolas Liochon, Steve Loughran, Lee Mallabone, Nathan Marz, Caitie McCaffrey, Josie McLellan, Christopher Meiklejohn, Ian Meyers, Neha Narkhede, Neha Narula, Cathy O'Neil, Onora O'Neill, Ludovic Orban, Zoran Perkov, Julia Powles, Chris Riccomini, Henry Robinson, David Rosenthal, Jennifer Rullmann, Matthew Sackman, Martin Scholl, Amit Sela, Gwen Shapira, Greg Spurrier, Sam Stokes, Ben Stopford, Tom Stuart, Diana Vasile, Rahul Vohra, Pete Warden, and Brett Wooldridge.

Several more people provided valuable feedback on drafts of the first edition: in particular, Raul Agepati, Tyler Akidau, Mattias Andersson, Sasha Baranov, Veena Basavaraj, David Beyer, Jim Brikman, Paul Carey, Raul Castro Fernandez, Joseph Chow, Derek Elkins, Sam Elliott, Alexander Gallego, Mark Grover, Stu Halloway, Heidi Howard, Nicola Kleppmann, Stefan Kruppa, Bjorn Madsen, Sander Mak, Stefan Podkowinski, Phil Potter, Hamid Ramazani, Sam Stokes, and Ben Summers.

Martin Kleppmann is also grateful for financial support he received during the writing of this book. While working on the first edition, he was given paid time to work on the book by LinkedIn, and later he was supported by a grant from The Boeing Company. During the writing of the second edition he was supported by a Freigeist Fellowship from the Volkswagen Foundation, by crowdfunding supporters including Ably, Mintter, Prisma, and SoftwareMill, and by sponsorship from ScyllaDB.

We would like to thank the team at O'Reilly for helping make this book real, and our colleagues and families for tolerating the vast amounts of time we have sunk into writing. We hope you find that it was worthwhile.

Chapter 1. Trade-Offs in Data Systems Architecture

> *There are no solutions; there are only trade-offs. […] But you try to get the best trade-off you can get, and that's all you can hope for.*
>
> —Thomas Sowell, interview with Fred Barnes (2005)

Data is central to much application development today. With web and mobile apps, software as a service (SaaS), and cloud services, it has become normal to store data from many different users in a shared server-based data infrastructure. Data from user activity, business transactions, devices, and sensors needs to be stored and made available for analysis. As users interact with an application, they both read the data that is stored and generate more data.

Small amounts of data, which can be stored and processed on a single machine, are often fairly easy to deal with. However, as the data volume or the rate of queries grows, it needs to be distributed across

multiple machines, which introduces many challenges. As the needs of the application become more complex, it is no longer sufficient to store everything in one system, and it might be necessary to combine multiple storage or processing systems that provide different capabilities.

We call an application *data-intensive* if data management is one of the primary challenges in developing the application [1]. While in *compute-intensive* systems the challenge is parallelizing a very large computation, in data-intensive applications we usually worry more about things like storing and processing large data volumes, managing changes to data, ensuring consistency in the face of failures and concurrency, and making sure services are highly available.

Such applications are typically built from standard building blocks that provide commonly needed functionality. For example, many applications need to do the following:

- Store data so that they, or another application, can find it again later (*databases*)
- Remember the result of an expensive operation, to speed up reads (*caches*)
- Allow users to search data by keyword or filter it in various ways (*search indexes*)
- Handle events and data changes as soon as they occur (*stream processing*)
- Periodically crunch a large amount of accumulated data (*batch processing*)

In building an application we typically take several software systems or services, such as databases or APIs, and glue them together with application code. If you are doing exactly what the data systems were designed for, this process can be quite easy.

However, as your application becomes more ambitious, challenges arise. There are many database systems with different characteristics, suitable for different purposes—how do you choose which one to use? There are various approaches to caching, several ways of building search indexes, and so on—how do you reason about their trade-offs? You need to figure out which tools and which approaches are the most appropriate for the task at hand, and it can be difficult to combine tools when you need to do something that a single tool cannot do alone.

This book is a guide to help you make decisions about which technologies to use and how to combine them. As you will see, no one approach is fundamentally better than others; everything has pros and cons. With this book, you will learn to ask the right questions to evaluate and compare data systems so that you can figure out which approach will best serve the needs of your particular application.

We will start our journey by looking at some of the ways that data is typically used in organizations today. Many of the ideas here have their origin in *enterprise software* (i.e., the software needs and engineering practices of large organizations, such as big corporations and governments), since historically, only large organizations had the large data volumes that required sophisticated technical solutions. If your data volume is small enough, you can simply keep it in a spreadsheet! However, more

recently it has also become common for smaller companies and startups to manage large data volumes and build data-intensive systems.

One of the key challenges with data systems is that different people need to do very different things with data. If you are working at a company, you and your team will have one set of priorities, while another team may have entirely different goals, even though you might be working with the same dataset! Moreover, those goals might not be explicitly articulated, which can lead to misunderstandings and disagreement about the right approach.

To help you understand your choices, this chapter compares several contrasting concepts and explores their trade-offs. We will consider the following topics:

- The difference between operational and analytical systems ("Operational Versus Analytical Systems")
- The pros and cons of cloud services and self-hosted systems ("Cloud Versus Self-Hosting")
- When to move from single-node systems to distributed systems ("Distributed Versus Single-Node Systems")
- Balancing the needs of the business and the rights of the user ("Data Systems, Law, and Society")

This chapter also defines terminology that you will need for the rest of the book.

TERMINOLOGY: FRONTENDS AND BACKENDS

Much of what we will discuss in this book relates to *backend development*. To explain that term: for web applications, the client-side code (which runs in a web browser) is called the *frontend*, and the server-side code that handles user requests is known as the *backend*. Mobile apps are similar to frontends in that they provide user interfaces, which often communicate over the internet with a server-side backend. Frontends sometimes manage data locally on the user's device [2], but the greatest data infrastructure challenges commonly lie in the backend: a frontend needs to handle only one user's data, whereas the backend manages data on behalf of *all* the users.

A backend service is often reachable via HTTP (or sometimes WebSocket); it usually consists of application code that reads and writes data in one or more databases and sometimes interfaces with additional data systems, such as caches or message queues (which we might collectively call *data infrastructure*). The application code is often *stateless* (i.e., when it finishes handling one HTTP request, it forgets everything about that request), and any information that needs to persist from one request to another needs to be stored either on the client or in the server-side data infrastructure.

Operational Versus Analytical Systems

If you are working on data systems in an enterprise, you are likely to encounter several different types of people who work with data. The first type are *backend engineers* who build services that handle requests for reading and updating data; these services often serve external users, either directly or indirectly via other services (see "Microservices and Serverless"). Sometimes services are for internal use by other parts of the organization.

In addition to the teams managing backend services, two other groups of people typically require access to an organization's data: *business analysts*, who generate reports about the activities of the organization to help management make better decisions (*business intelligence*, or BI), and *data scientists*, who look for novel insights in data or who create user-facing product features that are enabled by data analysis and machine learning (ML)/AI (e.g., "people who bought *X* also bought *Y*" recommendations on an ecommerce website, predictive analytics such as risk scoring or spam filtering, and ranking of search results).

Although business analysts and data scientists tend to use different tools and operate in different ways, they have some practices in common. First, both perform *analytics*, which means they look at the data that the users and backend services have generated. Second, they generally do not modify this data (except perhaps for fixing mistakes), although they might create derived datasets in which the original data has been processed in some way.

This has led to a split between two types of systems—a distinction that we will use throughout this book:

- *Operational systems* consist of the backend services and data infrastructure where data is created—for example, by serving external users. Here, the application code both reads and modifies the data in its databases, based on the actions performed by the users.
- *Analytical systems* serve the needs of business analysts and data scientists. They contain a read-only copy of the data from the operational systems, and they are optimized for the types of data processing that are needed for analytics.

As we shall see in the next section, operational and analytical systems are often kept separate, for good reasons. As these systems have matured, two new specialized roles have emerged: data engineers and analytics engineers. *Data engineers* are the people who know how to integrate the operational and analytical systems and who take responsibility for the organization's data infrastructure more widely [3]. *Analytics engineers* model and transform data to make it more useful for the business analysts and data scientists in an organization [4].

Many engineers specialize in either the operational or the analytical side. However, this book covers both operational and analytical data systems, since both play an important role in the lifecycle of data within an organization. We will explore in depth the data infrastructure that is used to deliver services

to both internal and external users so that you can work better with your colleagues on the other side of this divide.

Characterizing Transaction Processing and Analytics

In the early days of business data processing, a write to the database typically corresponded to a commercial transaction taking place: making a sale, placing an order with a supplier, paying an employee's salary, etc. As databases expanded into areas that didn't involve money changing hands, the term *transaction* nevertheless stuck, referring to a group of reads and writes that form a logical unit.

> **NOTE**
>
> Chapter 8 explores in detail what we mean by a transaction. This chapter uses the term loosely to refer to low-latency reads and writes.

Even though databases started being used for many kinds of data—posts on social media, moves in a game, contacts in an address book, and much, much more—the basic access pattern remained similar to processing business transactions. An operational system typically looks up a small number of records by a key (this is called a *point query*). Records are inserted, updated, or deleted based on the user's input. Because these applications are interactive, this access pattern became known as *online transaction processing* (OLTP).

However, databases also started being increasingly used for analytics, which has very different access patterns compared to OLTP. Usually, an analytical query scans over a huge number of records and calculates aggregate statistics (such as count, sum, or average) rather than returning the individual records to the user. For example, a business analyst at a supermarket chain may want to answer analytical queries such as these:

- What was the total revenue of each of our stores in January?
- How many more bananas than usual did we sell during our latest promotion?
- Which brand of baby food is most often purchased together with brand *X* diapers?

The reports that result from these types of queries are important for BI, helping management decide what to do next. To differentiate this pattern of using databases from transaction processing, it has been called *online analytical processing* (OLAP) [5]. The difference between OLTP and analytics is not always clear-cut, but some typical characteristics are listed in Table 1-1.

Table 1-1. Comparing characteristics of operational and analytical systems

Property	Operational systems (OLTP)	Analytical systems (OLAP)

Main read pattern	Point queries (fetch individual records by key)	Aggregate over large number of records
Main write pattern	Create, update, and delete individual records	Bulk import (ETL) or event stream
Human user example	End user of web/mobile application	Internal analyst, for decision support
Machine use example	Checking if an action is authorized	Detecting fraud/abuse patterns
Type of queries	Fixed, predefined by application	Arbitrary, ad-hoc exploration by analysts
Query volume	Lots of small queries	Few queries, each is complex
Data represents	Latest state of data (current point in time)	History of events that happened over time
Dataset size	Gigabytes to terabytes	Terabytes to petabytes

NOTE

The meaning of *online* in *OLAP* is unclear; it probably indicates that queries are not just for predefined reports, but that analysts use the OLAP system interactively for explorative queries.

With operational systems, users are generally not allowed to construct custom SQL queries and run them on the database, since that would potentially allow them to read or modify data that they do not have permission to access. They might also write queries that are expensive to execute and hence affect the database performance for other users. For these reasons, OLTP systems mostly run fixed sets of queries that are baked into the application code, with one-off custom queries used only occasionally for maintenance or troubleshooting. On the other hand, analytical databases usually give their users the freedom to write arbitrary SQL queries by hand, or to generate queries automatically using a data visualization or dashboard tool such as Tableau, Looker, or Microsoft Power BI.

Another type of system is designed for analytical workloads (queries that aggregate over many records) but embedded into user-facing products. Systems designed for this type of use, known as *product analytics* or *real-time analytics*, include Pinot, Druid, and ClickHouse [6]. Such systems ingest data in

real time and are optimized for low-latency query responses. In contrast, traditional OLAP systems typically ingest data in batches and are optimized for high-throughput query processing.

Data Warehousing

At first, the same databases were used for both transaction processing and analytical queries. SQL turned out to be quite flexible in this regard; it works well for both types of queries. In the late 1980s and early 1990s, however, a trend arose for companies to stop using their OLTP systems for analytics purposes and to run the analytics on a separate database system instead. This separate database was called a *data warehouse*.

A large enterprise may have dozens, even hundreds, of OLTP systems: systems powering the customer-facing website, controlling point-of-sale (checkout) systems in physical stores, tracking inventory in warehouses, planning routes for vehicles, managing suppliers, administering employees, and performing many other tasks. Each of these systems is complex and needs a team of people to maintain it, so they end up operating mostly independently from one another.

It is usually undesirable for business analysts and data scientists to directly query these OLTP systems, for several reasons:

- The data of interest may be spread across multiple operational systems, making it difficult to combine those datasets in a single query (a problem known as *data silos*).
- The kinds of schemas and data layouts that are good for OLTP are less well suited for analytics (see "Stars and Snowflakes: Schemas for Analytics").
- Analytical queries can be quite expensive, and running them on an OLTP database would impact the performance for other users.
- The OLTP systems might reside in a separate network that users are not allowed to directly access, for security or compliance reasons.

A *data warehouse*, by contrast, is a separate database that analysts can query to their hearts' content, without affecting OLTP operations [7]. As we shall see in Chapter 4, data warehouses often store data very differently from OLTP databases, to optimize for the types of queries that are common in analytics.

The data warehouse contains a read-only copy of the data from all the various OLTP systems in the company. Data is extracted from OLTP databases (using either a periodic data dump or a continuous stream of updates), transformed into an analysis-friendly schema, cleaned up, and then loaded into the data warehouse. This process of getting data into the data warehouse is known as *extract–transform–load* (ETL) and is illustrated in Figure 1-1. Sometimes the order of the *transform* and *load* steps is swapped (i.e., the transformation is done in the data warehouse, after loading), resulting in *ELT*.